

JavaScript

**: Consumindo
e tratando
dados de uma
API**



Assincronicidad e

Callback

s



Eu estava com fome e coloquei uma lasanha no microondas por 5 minutos. Após esse tempo, eu farei uma ação, que nesse caso vai ser a de comer. Esse processo de espera é algo que podemos visualizar no javascript com os callbacks, que podem ser interpretados como ações que você faz que irão resultar em outra ação.



5min



`almoço(esquentarLasanha)`

`almoço()`

`almoço(esquentarLasanha), 5000`

O callback é uma função que é passada como argumento para uma outra função (higher-order function) e, geralmente, só é executada quando alguma operação é concluída ou quando um evento específico ocorre. O cenário de uso mais comum é, certamente, quando as requisições são finalizadas.

almoço(esquentarLasanha, 5000)

(Callback)



(Higher-order
function)

almoço(**esquentarLasanha**)

almoço()

O callback é uma função que é passada como argumento para uma outra função (higher-order function) e, geralmente, só é executada quando alguma operação é concluída ou quando um evento específico ocorre. O cenário de uso mais comum é, certamente, quando as requisições são finalizadas.

almoço(esquentarLasanha, 5000)

(Callback)

(Higher-order
function)

almoço(esquentarLasanha)

```
setTimeout(mandaMensagem, 5000);
```

almoço()

```
setTimeout();
```

Fetch API



Ao iniciarmos o trabalho, nos deparamos com um problema entre os dados de endereço dos usuários.



Os endereços estavam despadronizados:

Cliente 1	Rua Getúlio Vargas, número 1520
Cliente 2	R. G. Vargas, 1520
Cliente 3	R. Getúlio Vargas, 1520
Cliente 4	G. Vargas - n 1520

Como vamos resolver essa
situação?



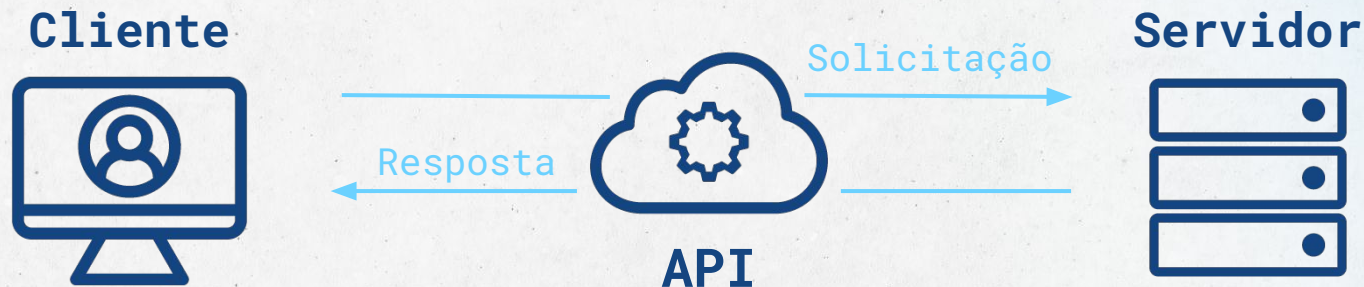
API

Interface de Programação de Aplicações

Mecanismos que permitem que dois componentes de software se comuniquem usando um conjunto de definições e protocolos.

API

Interface de Programação de Aplicações



API

Interface de Programação de Aplicações

Cliente



Solicitação



Pagar boleto

Servidor



Resposta

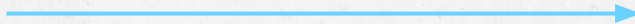
API

Interface de Programação de Aplicações

Cliente



Solicitação



API



Resposta

Servidor



VIA CEP

Vamos usar uma API chamada [ViaCEP](#).

Cliente



Requisição de CEP



API

Resposta com dados

Servidor



VIA CEP

API

Interface de Programação de Aplicações

JS script.js > ...

```
1  var consultaCEP = fetch('viacep.com.br/ws/01001000/json/');  
2  
3  console.log(consultaCEP);|
```

O método **Fetch** é assíncrono e tem como parâmetro obrigatório a URL da API. Ou seja, usamos o ViaCEP, usando outra opção, obteríamos outro link, e é esse link que é obrigatório, o único parâmetro obrigatório desse método.

API

Interface de Programação de Aplicações

```
script.js:3
▼ Promise ⓘ
  ► [[Prototype]]: Promise
    [[PromiseState]]: "rejected"
  ► [[PromiseResult]]: TypeError: Failed to fetch at file:///C:/User
```

Promise s



Promise

Promise

Definem uma ação que vai ser executada no futuro e ela pode ser resolvida (com sucesso) ou rejeitada (com erro).



Resolvida



Rejeitada

Promise



Imagine que você recebeu a notificação de que sua encomenda saiu para entrega. A notificação foi uma promessa feita pela transportadora.



Anatomia da Promise

```
const entrega = new Promise(function  
(resolve, reject) {  
  if (recebeu == true) {  
    resolve('Ana recebeu a encomenda!')  
  } else {  
    reject('Não foi possível receber a  
    encomenda!')  
  }  
})
```

Anatomia da Promise

▼ Promise ⓘ

▶ `[[Prototype]]: Promise`

`[[PromiseState]]: "rejected"`

▶ `[[PromiseResult]]: TypeError:`

Anatomia da Promise

```
index.html JS script.js X
JS script.js > [?] consultaCEP
1  var consultaCEP = fetch('https://viacep.com.br/ws/01001000/json/');
2
3  console.log(consultaCEP);
```

```
Promise {<pending>} script.js:3
  ▶ [[Prototype]]: Promise
  ▶ [[PromiseState]]: "fulfilled"
  ▶ [[PromiseResult]]: Response
>
```

Anatomia da Promise

```
▼ Promise {<pending>}  script.js:3  
  ► [[Prototype]]: Promise  
    [[PromiseState]]: "fulfilled"  
  ▼ [[PromiseResult]]: Response  
    body: (...)  
    bodyUsed: false  
    ► headers: Headers {}  
      ok: true  
      redirected: false  
      status: 200  
      statusText: "OK"  
      type: "cors"  
      url: "https://viacep.com.br/ws/01001000/json/"  
    ► [[Prototype]]: Response
```



Promise

